

```
In [1]: import pandas as pd
```

```
In [5]: movies = pd.read_csv(r'C:\Users\SID Balurkar\Desktop\Full Stack Data Science\Classwork\Nov 2025\25th Nov - Seaborn\Mc
movies
```

```
Out[5]:
```

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

```
In [6]: type(movies)
```

```
Out[6]: pandas.core.frame.DataFrame
```

```
In [7]: len(movies)
```

```
Out[7]: 559
```

```
In [8]: import numpy
```

```
print(numpy.__version__)
```

2.1.3

```
In [9]: import pandas  
print(pandas.__version__)
```

2.2.3

```
In [10]: movies.columns
```

```
Out[10]: Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %',  
               'Budget (million $)', 'Year of release'],  
              dtype='object')
```

```
In [11]: movies.info
```

```
Out[11]: <bound method DataFrame.info of
```

	Film	Genre	Rotten Tomatoes Ratings % \
0	(500) Days of Summer	Comedy	87
1	10,000 B.C.	Adventure	9
2	12 Rounds	Action	30
3	127 Hours	Adventure	93
4	17 Again	Comedy	55
..	...	...	...
554	Your Highness	Comedy	26
555	Youth in Revolt	Comedy	68
556	Zodiac	Thriller	89
557	Zombieland	Action	90
558	Zookeeper	Comedy	14

	Audience Ratings %	Budget (million \$)	Year of release
0	81	8	2009
1	44	105	2008
2	52	20	2009
3	84	18	2010
4	70	20	2009
..	...	...	...
554	36	50	2011
555	52	18	2009
556	73	65	2007
557	87	24	2009
558	42	80	2011

[559 rows x 6 columns]>

```
In [12]: movies.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Film                                  559 non-null    object
1   Genre                                559 non-null    object
2   Rotten Tomatoes Ratings %           559 non-null    int64
3   Audience Ratings %                  559 non-null    int64
4   Budget (million $)                  559 non-null    int64
5   Year of release                      559 non-null    int64
dtypes: int64(4), object(2)
memory usage: 26.3+ KB

```

```
In [13]: movies.shape
```

```
Out[13]: (559, 6)
```

```
In [14]: movies.head()
```

```
Out[14]:
```

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

```
In [15]: movies.tail()
```

```
Out[15]:
```

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million \$)	Year of release
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

```
In [16]: movies.columns
```

```
Out[16]: Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %',
               'Budget (million $)', 'Year of release'],
              dtype='object')
```

```
In [18]: movies.columns = ['Film', 'Genre', 'CriticRating', 'AudienceRating', 'BudgetMillions', 'Year']
movies.columns
```

```
Out[18]: Index(['Film', 'Genre', 'CriticRating', 'AudienceRating', 'BudgetMillions',
               'Year'],
              dtype='object')
```

```
In [19]: movies.head(1)
```

```
Out[19]:
```

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009

```
In [20]: movies.shape
```

```
Out[20]: (559, 6)
```

```
In [21]: movies.describe()
```

Out[21]:

	CriticRating	AudienceRating	BudgetMillions	Year
count	559.000000	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136	2009.152057
std	26.413091	16.826887	48.731817	1.362632
min	0.000000	0.000000	0.000000	2007.000000
25%	25.000000	47.000000	20.000000	2008.000000
50%	46.000000	58.000000	35.000000	2009.000000
75%	70.000000	72.000000	65.000000	2010.000000
max	97.000000	96.000000	300.000000	2011.000000

In [22]: `movies.describe().transpose()`

Out[22]:

	count	mean	std	min	25%	50%	75%	max
CriticRating	559.0	47.309481	26.413091	0.0	25.0	46.0	70.0	97.0
AudienceRating	559.0	58.744186	16.826887	0.0	47.0	58.0	72.0	96.0
BudgetMillions	559.0	50.236136	48.731817	0.0	20.0	35.0	65.0	300.0
Year	559.0	2009.152057	1.362632	2007.0	2008.0	2009.0	2010.0	2011.0

In [23]: `movies.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Film            559 non-null   object
1   Genre           559 non-null   object
2   CriticRating    559 non-null   int64
3   AudienceRating  559 non-null   int64
4   BudgetMillions  559 non-null   int64
5   Year            559 non-null   int64
dtypes: int64(4), object(2)
memory usage: 26.3+ KB

```

```
In [26]: movies.Film = movies.Film.astype('category')
```

```
In [29]: movies.describe()
```

```
Out[29]:
```

	CriticRating	AudienceRating	BudgetMillions	Year
count	559.000000	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136	2009.152057
std	26.413091	16.826887	48.731817	1.362632
min	0.000000	0.000000	0.000000	2007.000000
25%	25.000000	47.000000	20.000000	2008.000000
50%	46.000000	58.000000	35.000000	2009.000000
75%	70.000000	72.000000	65.000000	2010.000000
max	97.000000	96.000000	300.000000	2011.000000

```
In [30]: movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Film            559 non-null    category
1   Genre           559 non-null    object
2   CriticRating    559 non-null    int64
3   AudienceRating  559 non-null    int64
4   BudgetMillions  559 non-null    int64
5   Year            559 non-null    int64
dtypes: category(1), int64(4), object(1)
memory usage: 43.6+ KB
```

```
In [31]: movies.Genre = movies.Genre.astype('category')
        movies.Year = movies.Year.astype('category')
```

```
In [32]: movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Film            559 non-null    category
1   Genre           559 non-null    category
2   CriticRating    559 non-null    int64
3   AudienceRating  559 non-null    int64
4   BudgetMillions  559 non-null    int64
5   Year            559 non-null    category
dtypes: category(3), int64(3)
memory usage: 36.5 KB
```

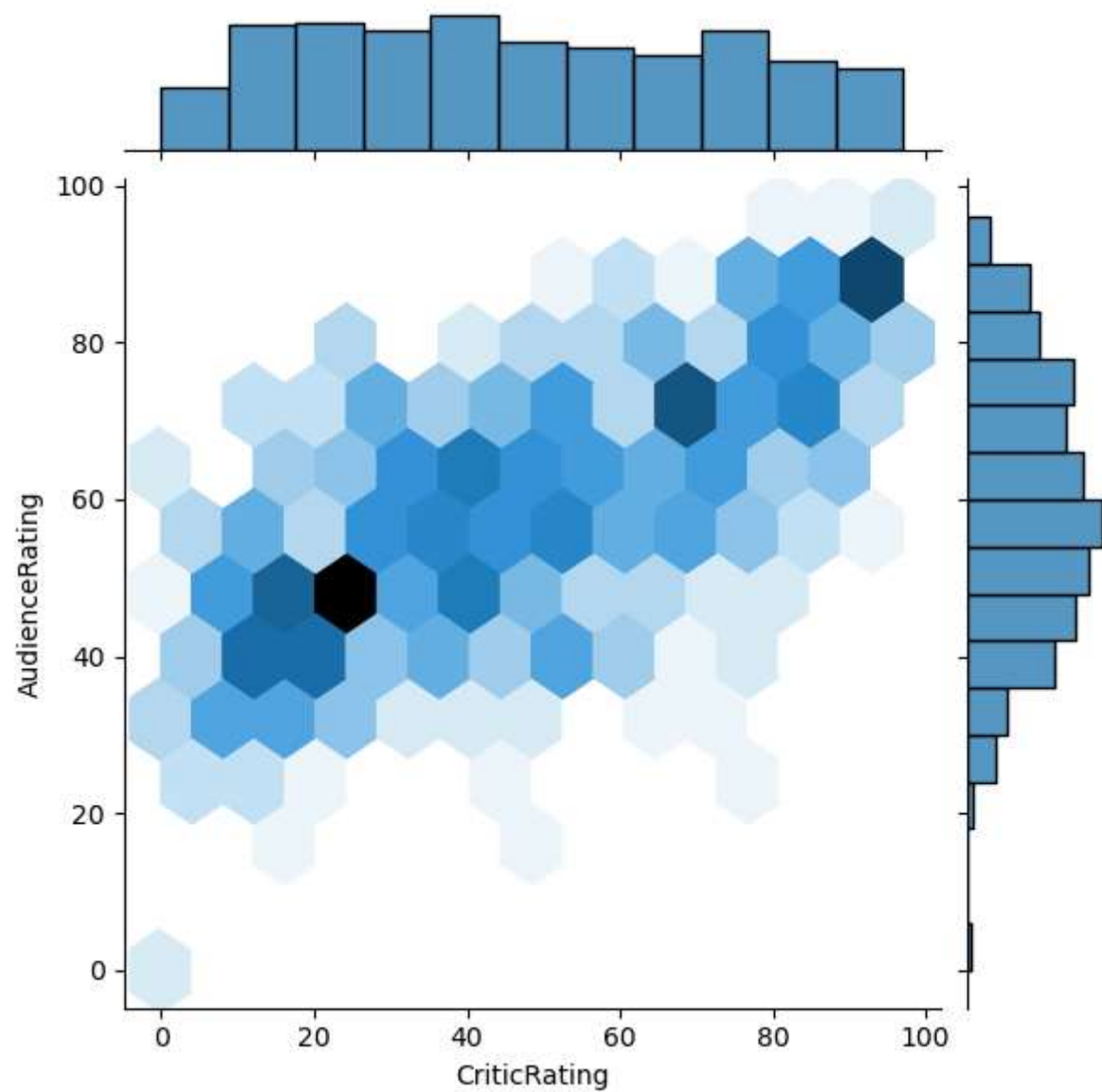
```
In [33]: movies.describe()
```


Out[33]:

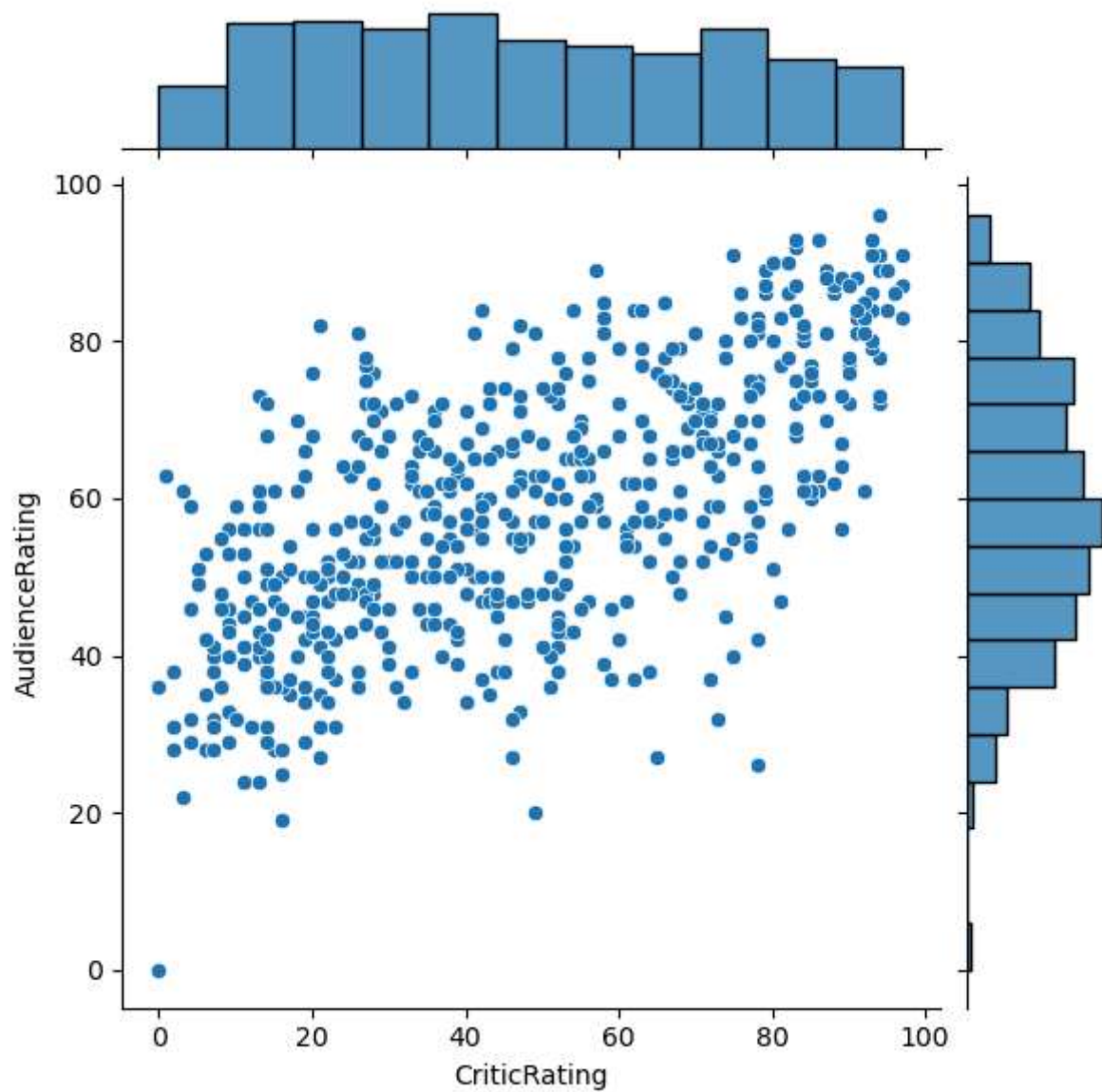
	CriticRating	AudienceRating	BudgetMillions
count	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136
std	26.413091	16.826887	48.731817
min	0.000000	0.000000	0.000000
25%	25.000000	47.000000	20.000000
50%	46.000000	58.000000	35.000000
75%	70.000000	72.000000	65.000000
max	97.000000	96.000000	300.000000

```
In [35]: import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

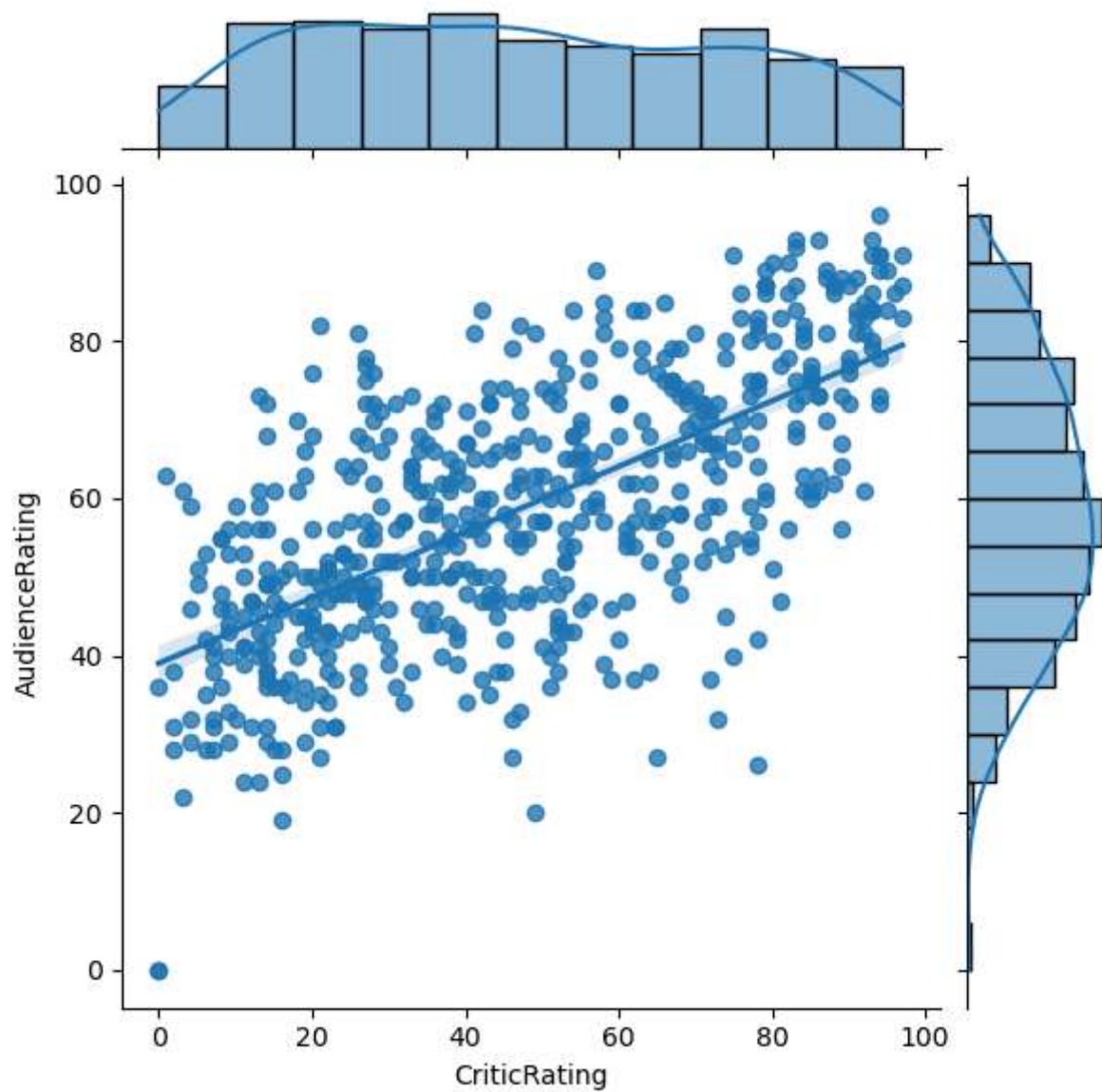
```
In [38]: j = sns.jointplot(data = movies, x = 'CriticRating', y = 'AudienceRating', kind='hex')
```



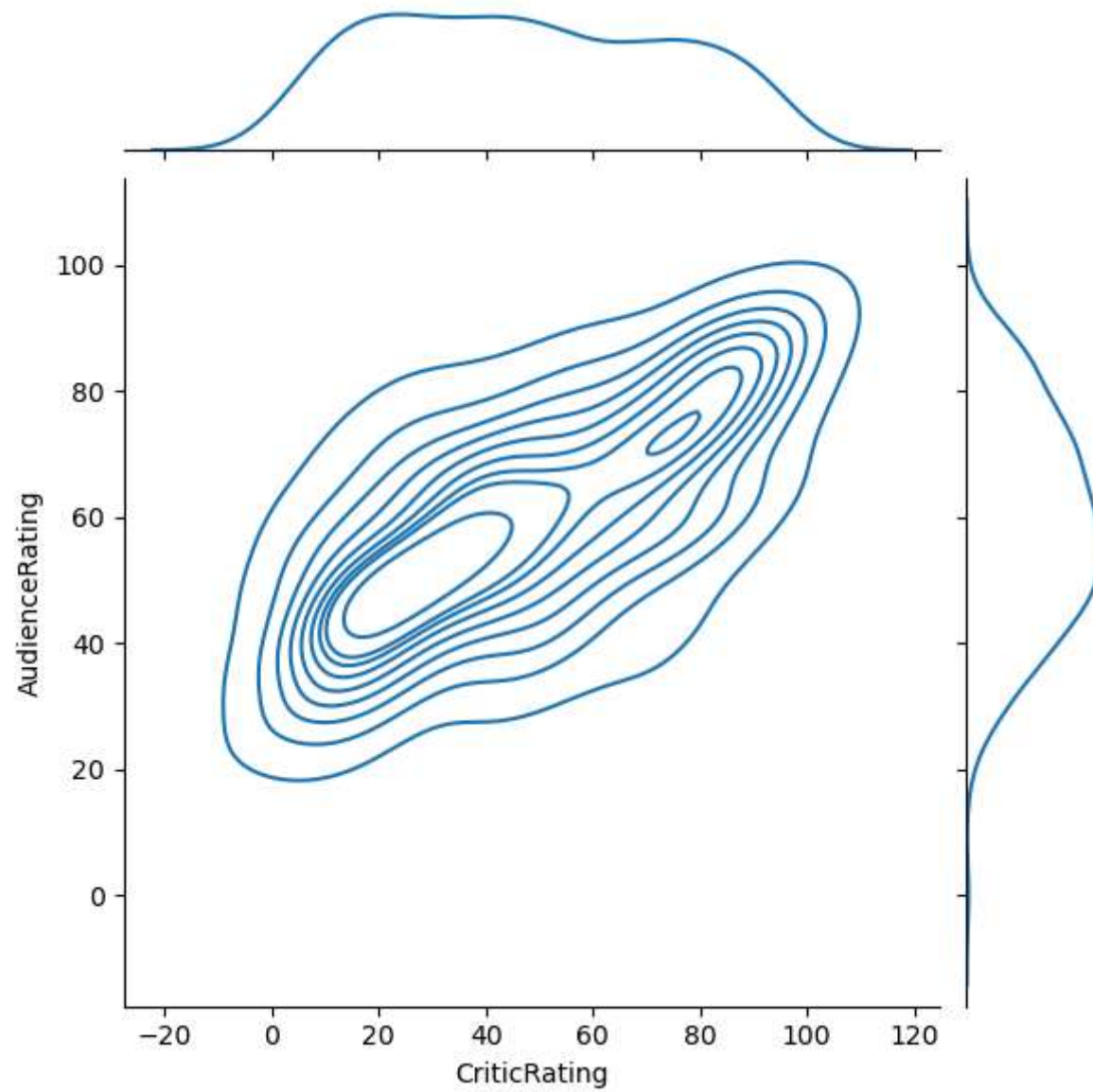
```
In [39]: j = sns.jointplot(data = movies, x = 'CriticRating', y = 'AudienceRating', kind='scatter')
```



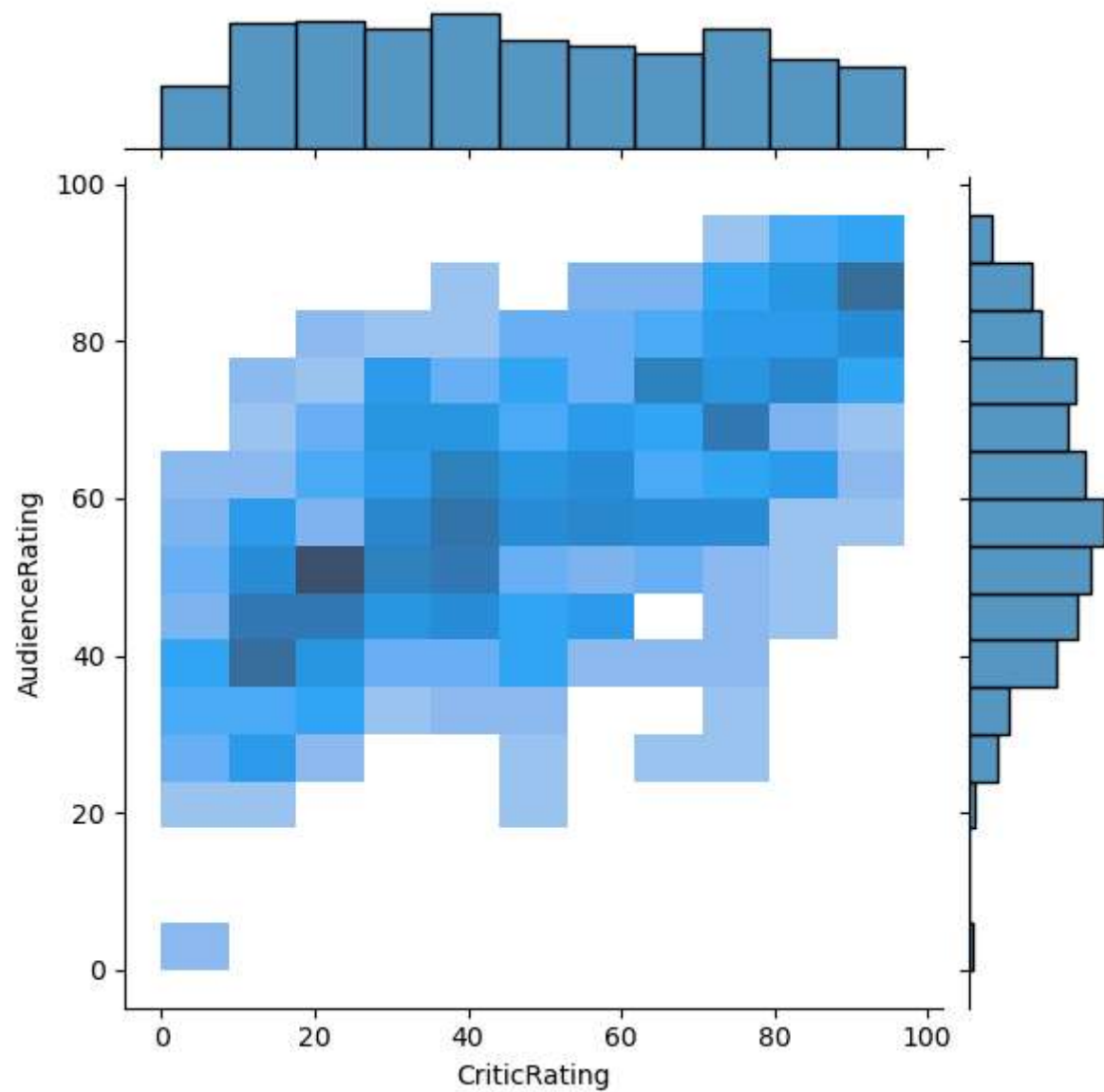
```
In [40]: j = sns.jointplot(data = movies, x = 'CriticRating', y = 'AudienceRating', kind='reg')
```



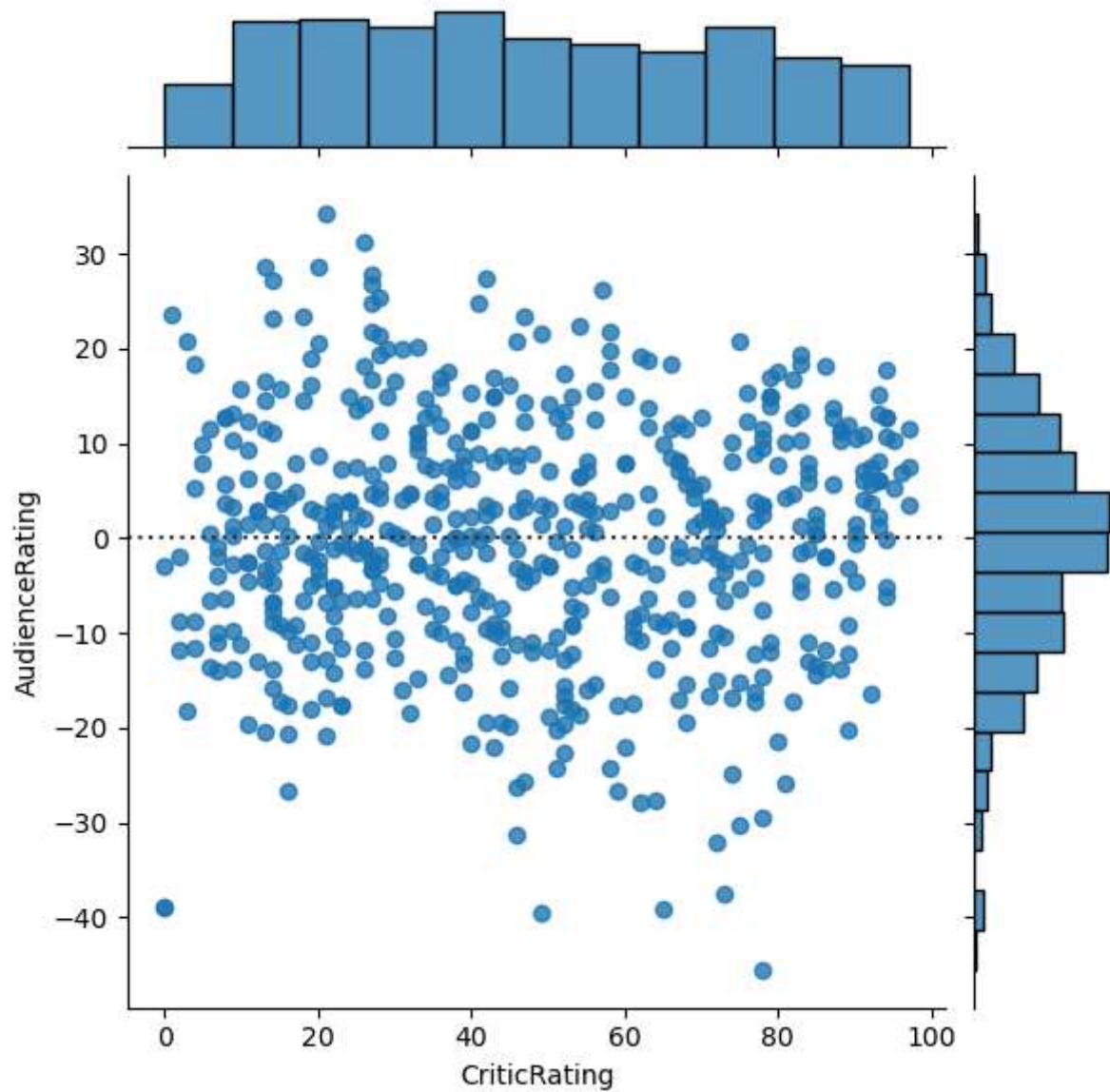
```
In [41]: j = sns.jointplot(data = movies, x = 'CriticRating', y = 'AudienceRating', kind='kde')
```



```
In [42]: j = sns.jointplot(data = movies, x = 'CriticRating', y = 'AudienceRating', kind='hist')
```

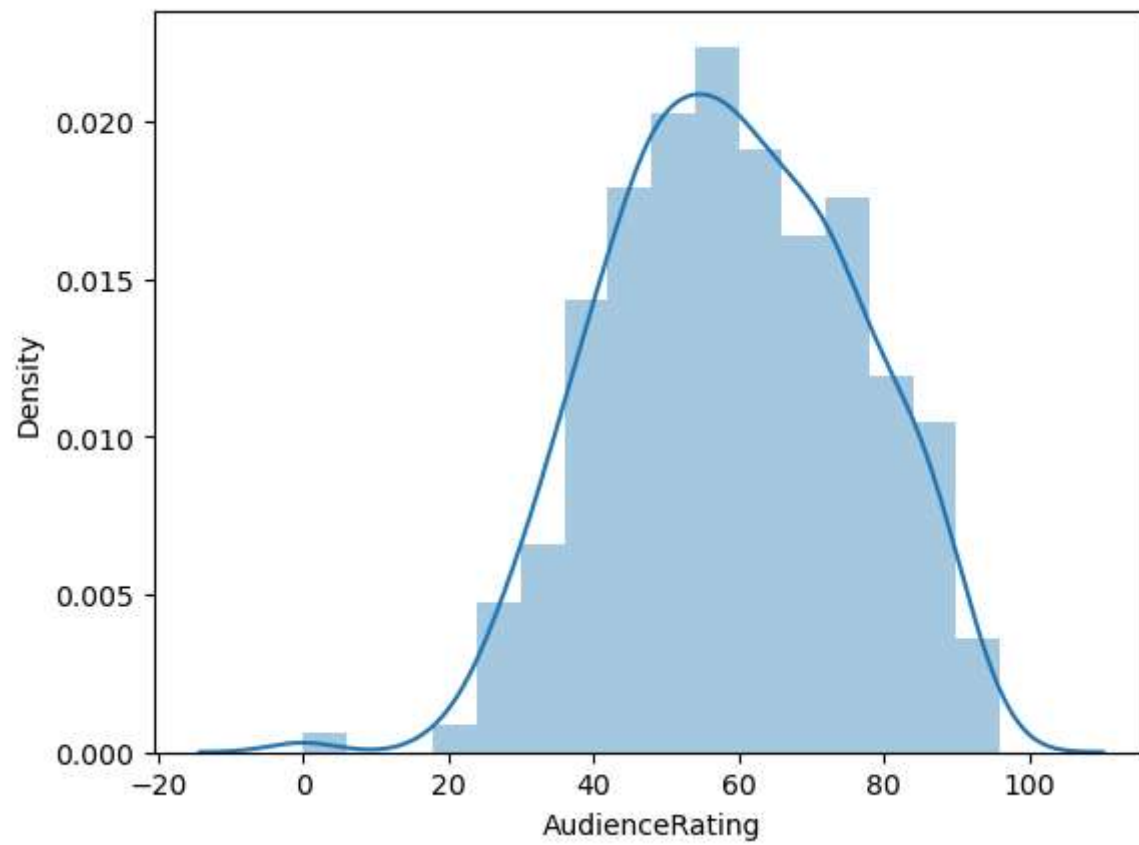


```
In [44]: j = sns.jointplot(data = movies, x = 'CriticRating', y = 'AudienceRating', kind='resid')
```

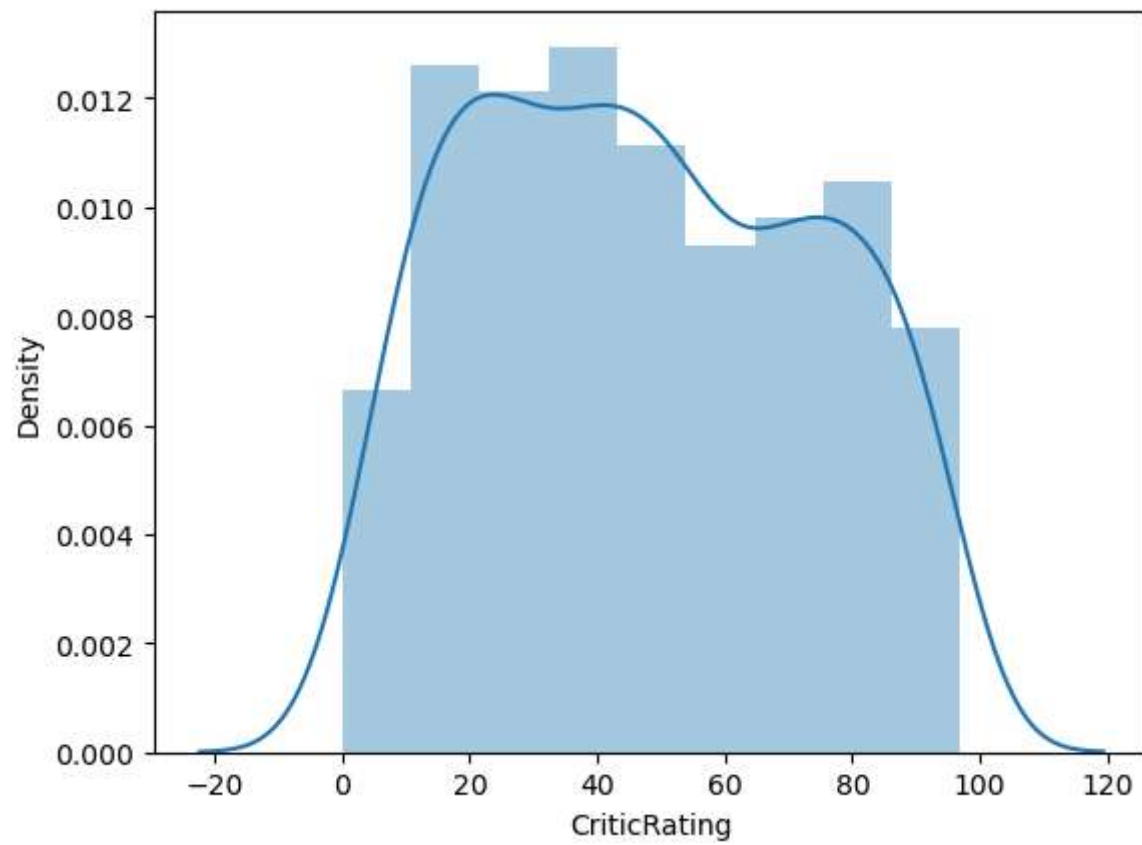


```
In [ ]: #Histograms  
        #Distribution plot
```

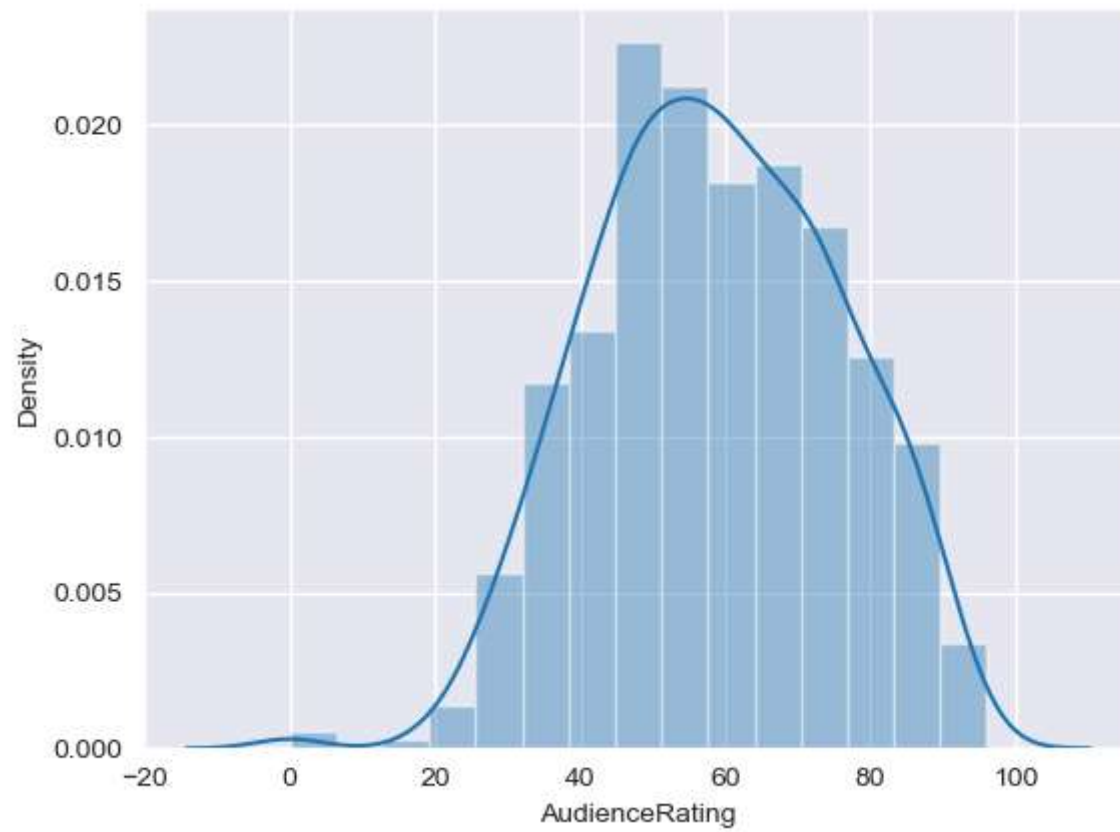
```
In [45]: m1 = sns.distplot(movies.AudienceRating)
```



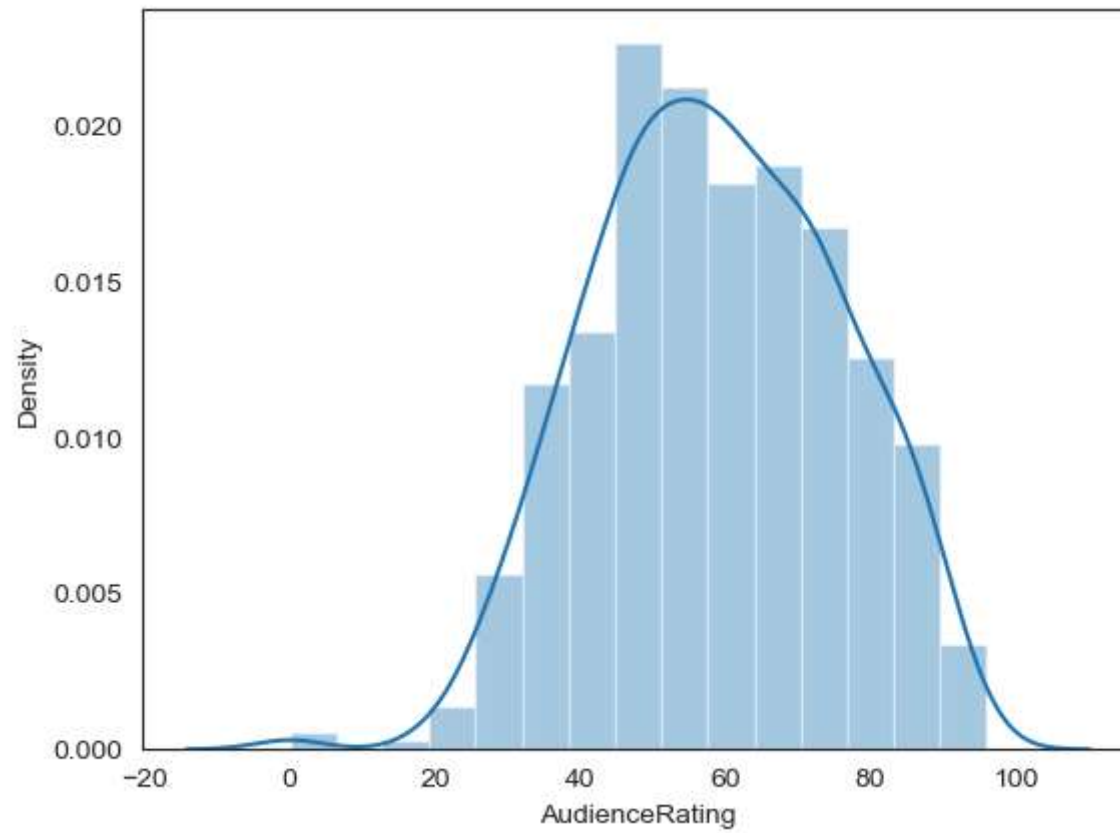
```
In [46]: m1 = sns.distplot(movies.CriticRating)
```

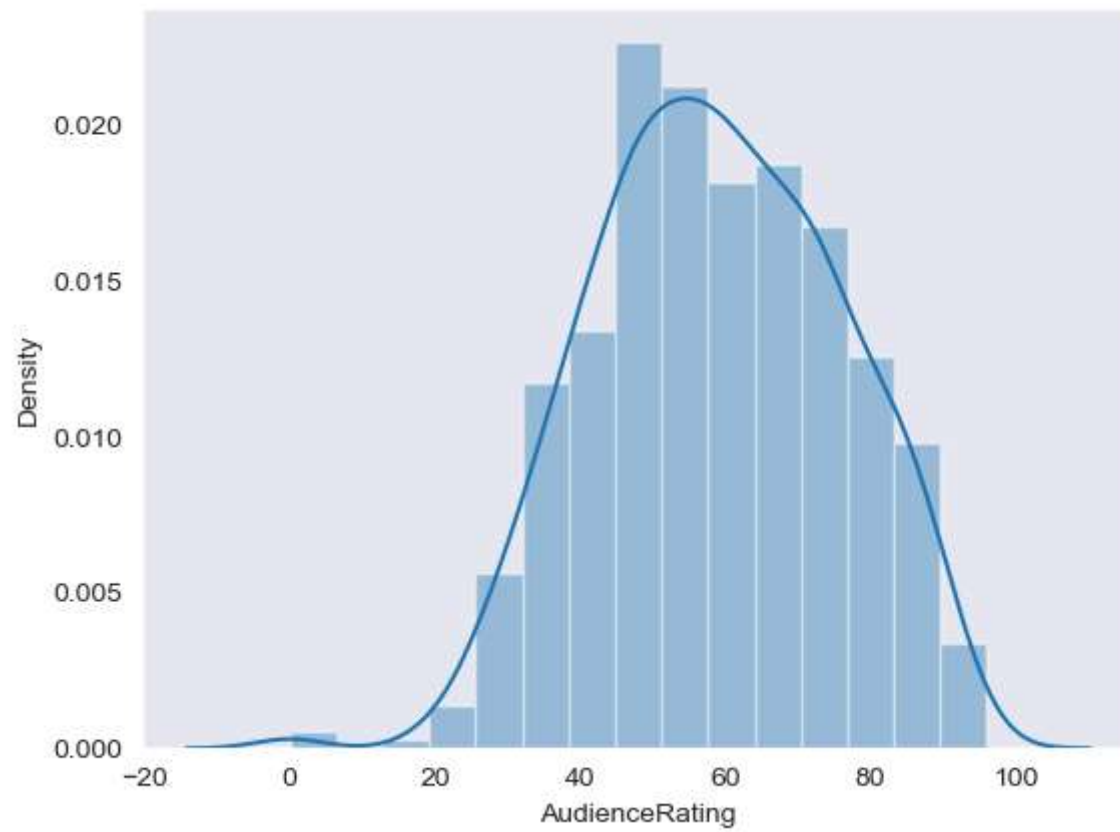
```
In [49]: sns.set_style('darkgrid')  
m2 = sns.distplot(movies.AudienceRating, bins=15)
```



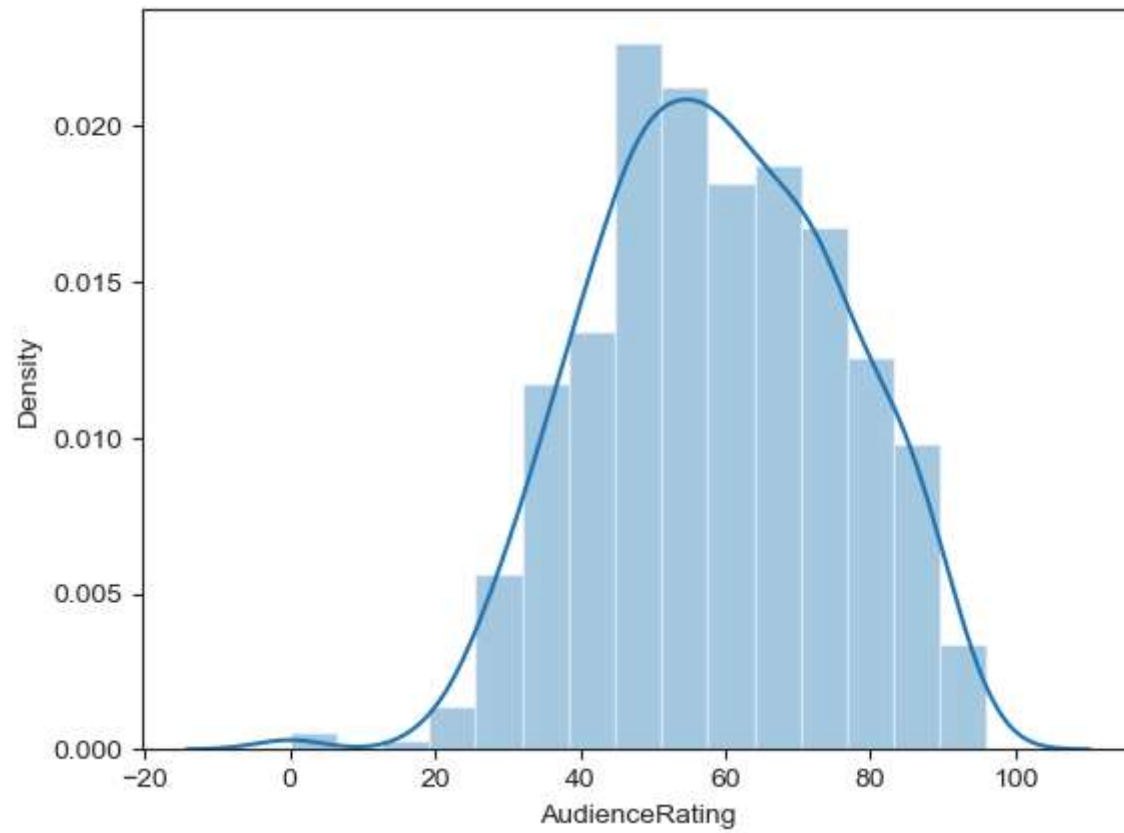
```
In [50]: sns.set_style('white')
m3 = sns.distplot(movies.AudienceRating, bins=15)
```



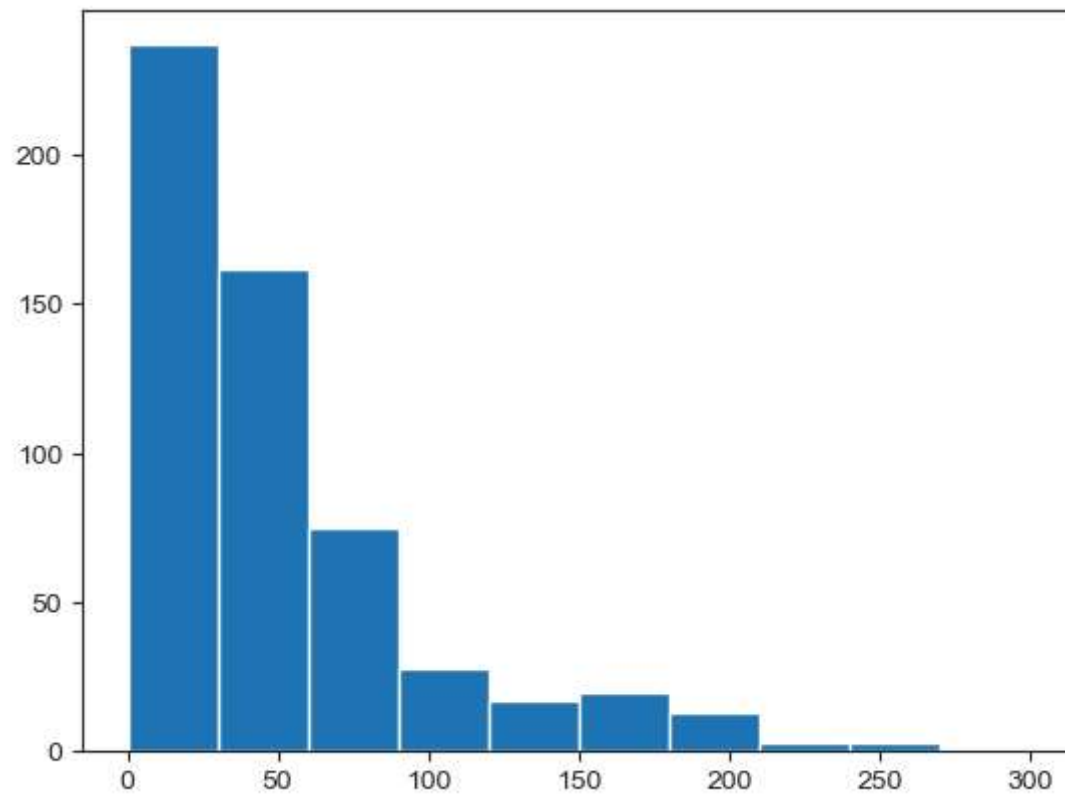
```
In [51]: sns.set_style('dark')
m2 = sns.distplot(movies.AudienceRating, bins=15)
```



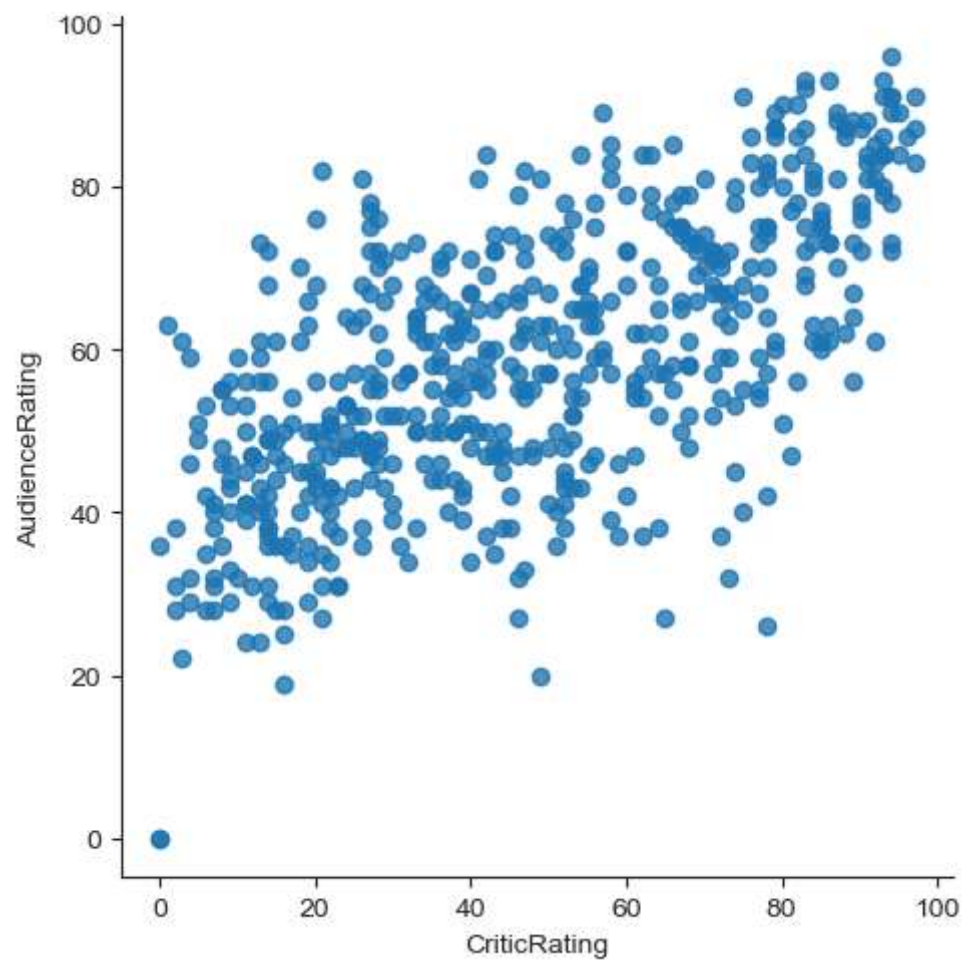
```
In [52]: sns.set_style('ticks')
m4 = sns.distplot(movies.AudienceRating, bins=15)
```



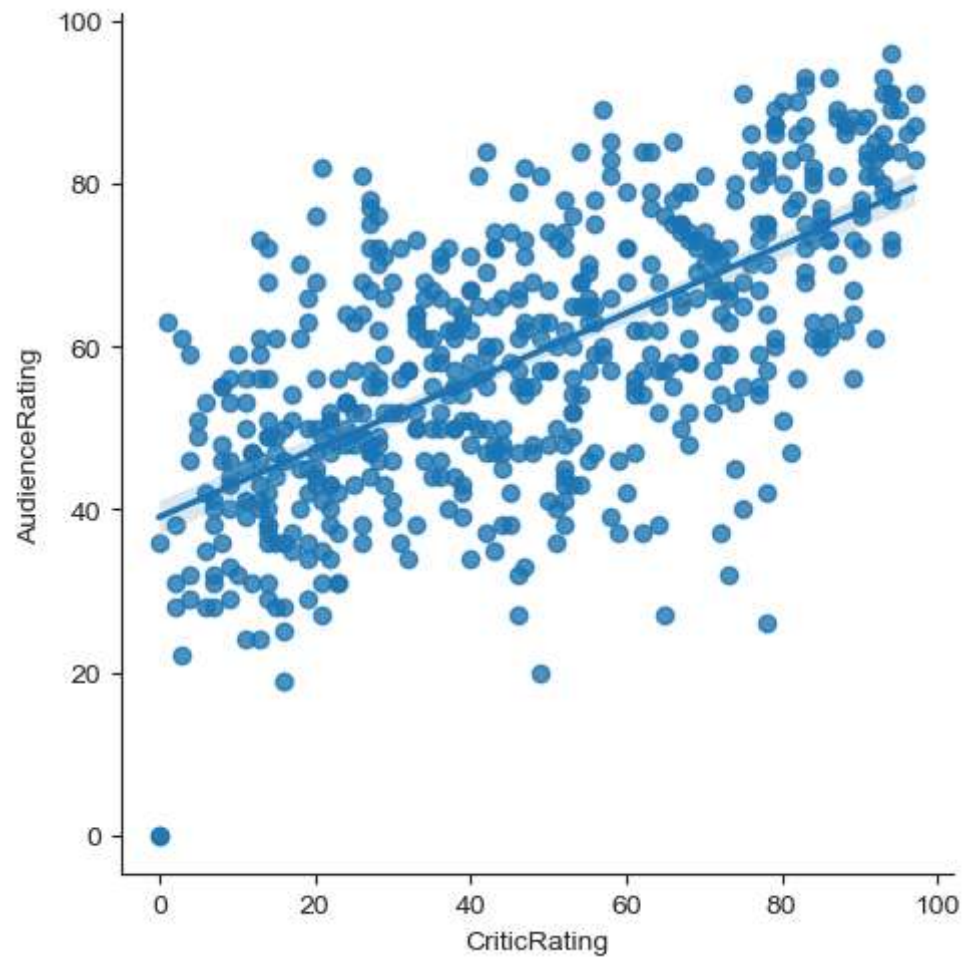
```
In [53]: plt.hist(movies.BudgetMillions)
plt.show()
```



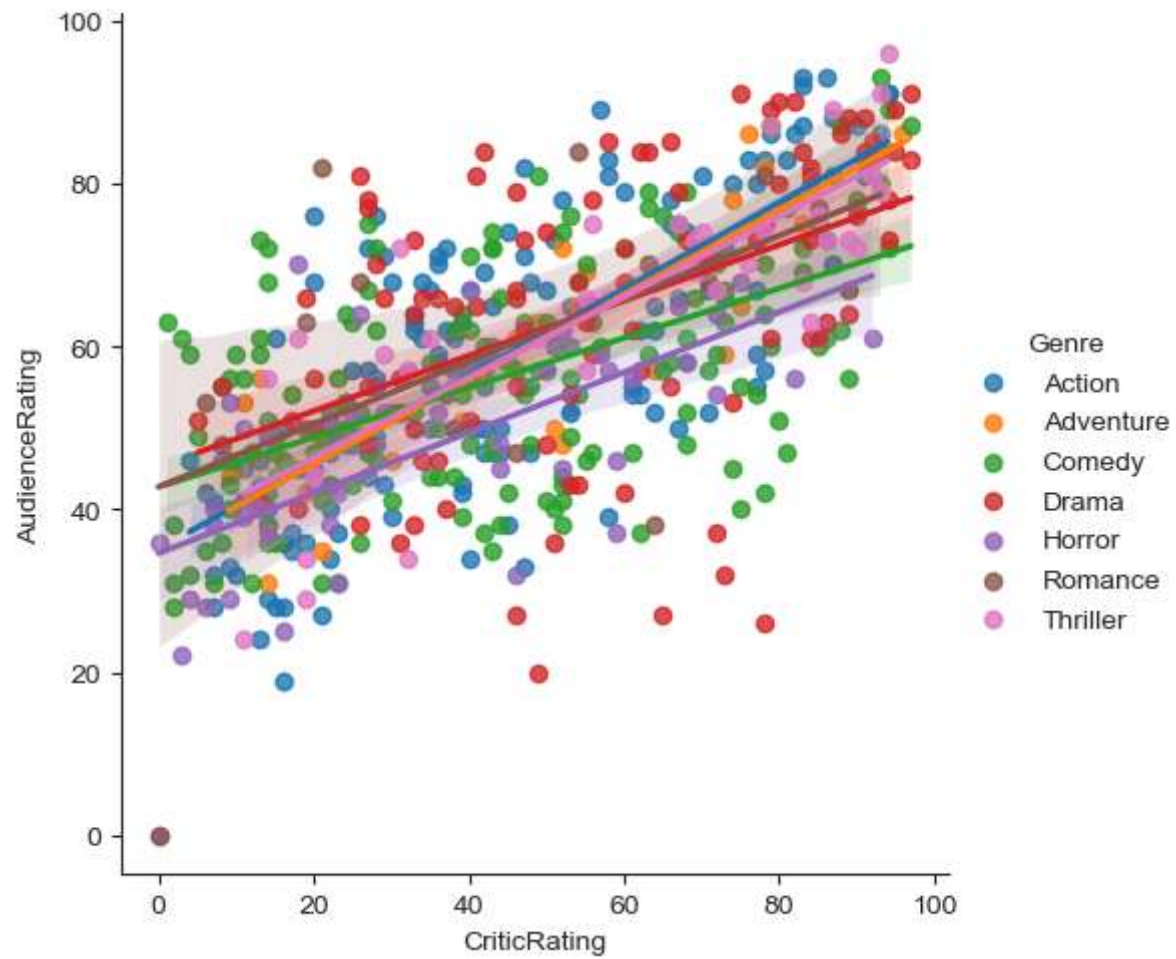
```
In [54]: vis1 = sns.lmplot(data = movies, x = 'CriticRating', y = 'AudienceRating', fit_reg = False)
```



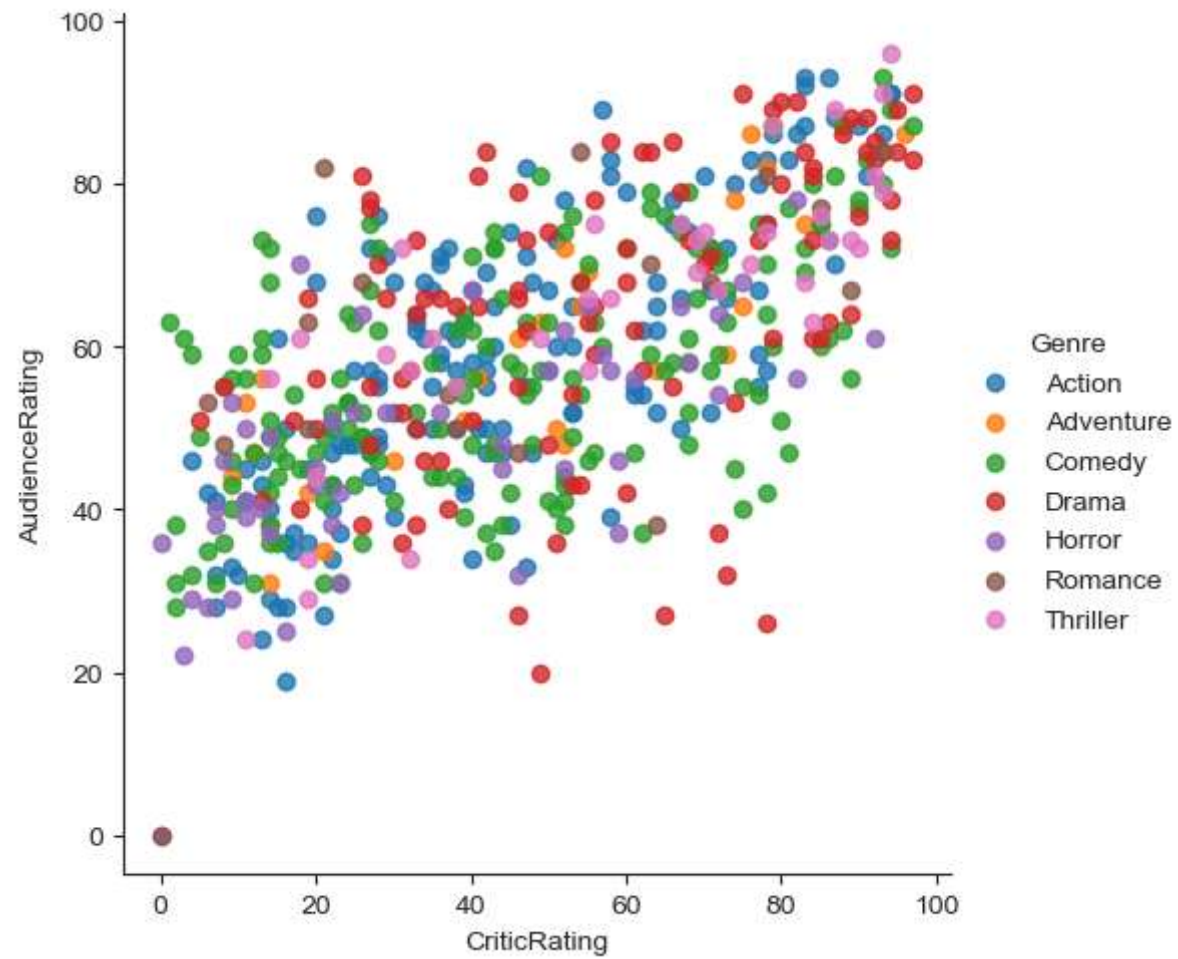
```
In [55]: vis1 = sns.lmplot(data = movies, x = 'CriticRating', y = 'AudienceRating', fit_reg = True)
```



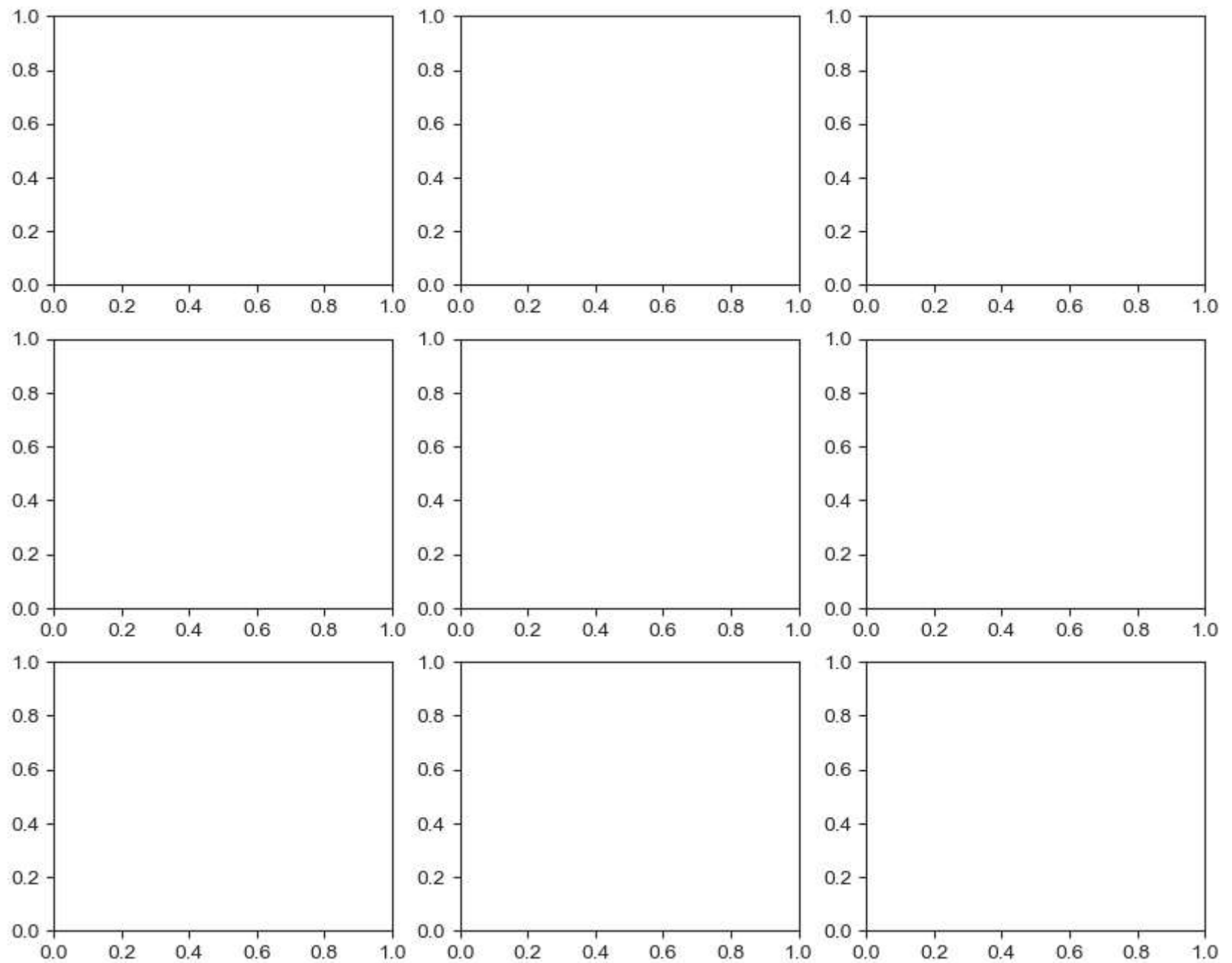
```
In [56]: vis1 = sns.lmplot(data = movies, x = 'CriticRating', y = 'AudienceRating', fit_reg = True, hue = 'Genre')
```

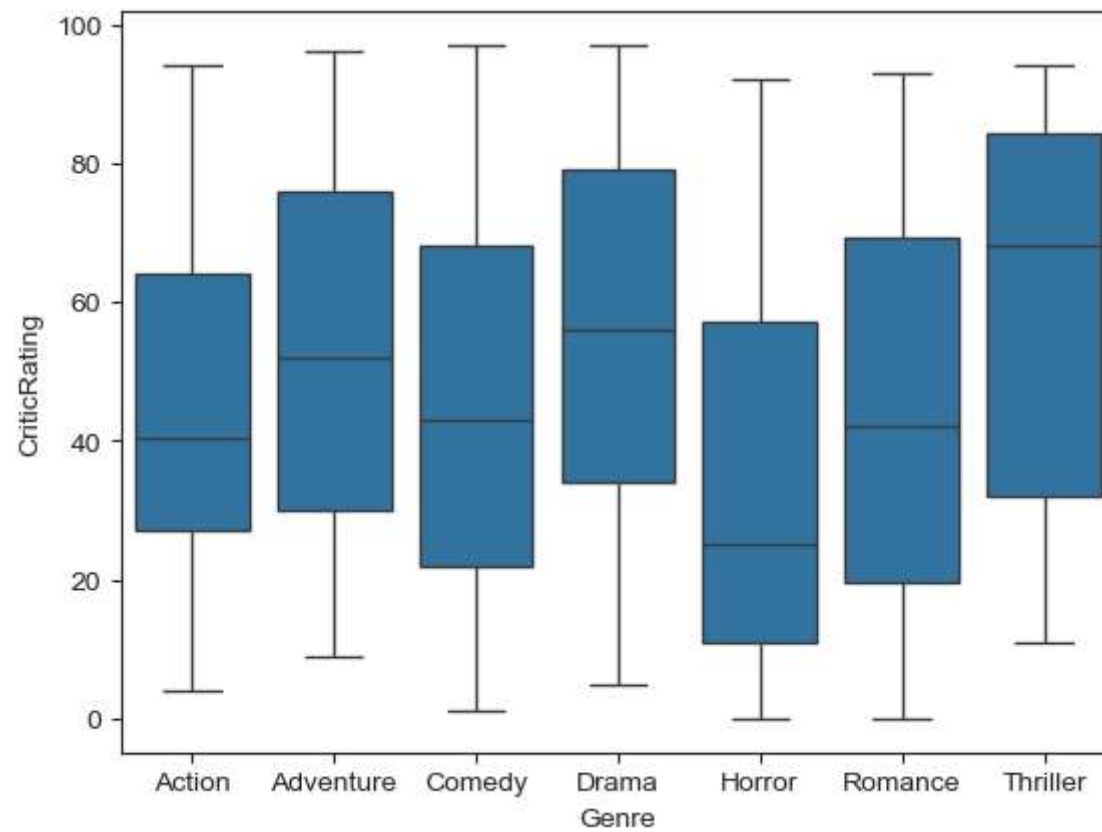
```
In [57]: vis1 = sns.lmplot(data = movies, x = 'CriticRating', y = 'AudienceRating', fit_reg = False, hue = 'Genre')
```



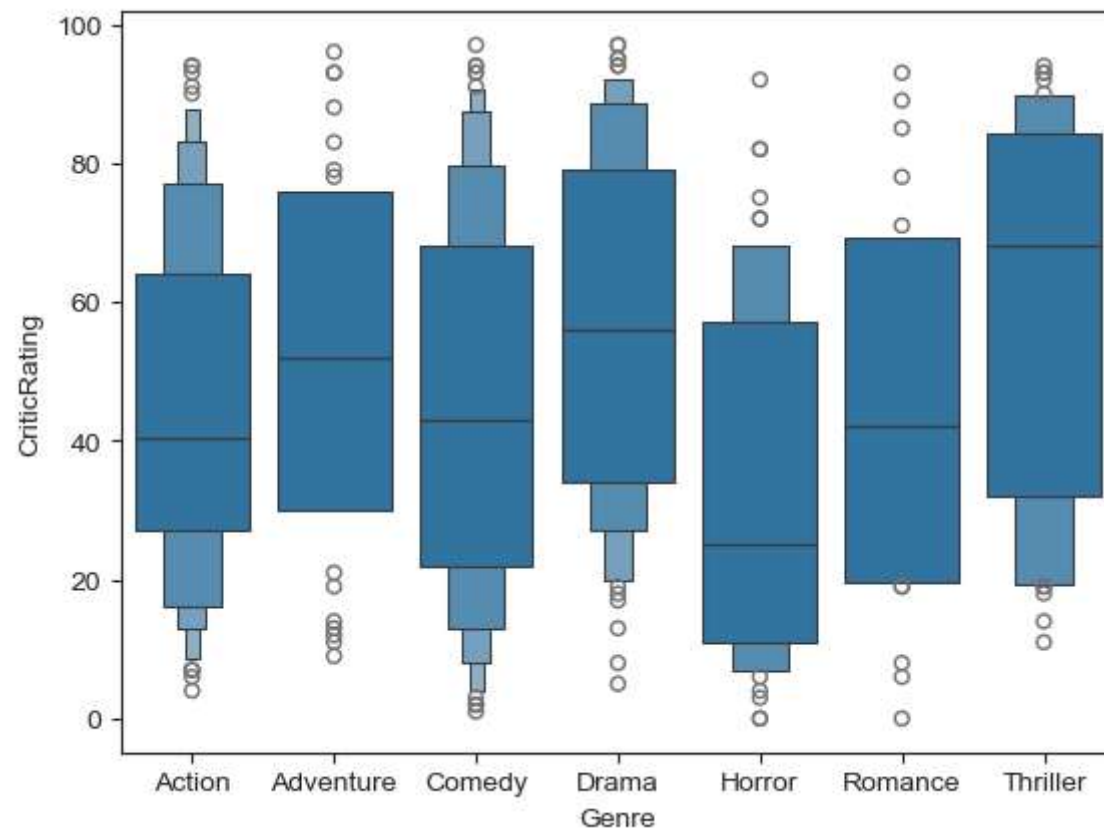
```
In [63]: ax = plt.subplots(3,3,figsize=(10,8))
```



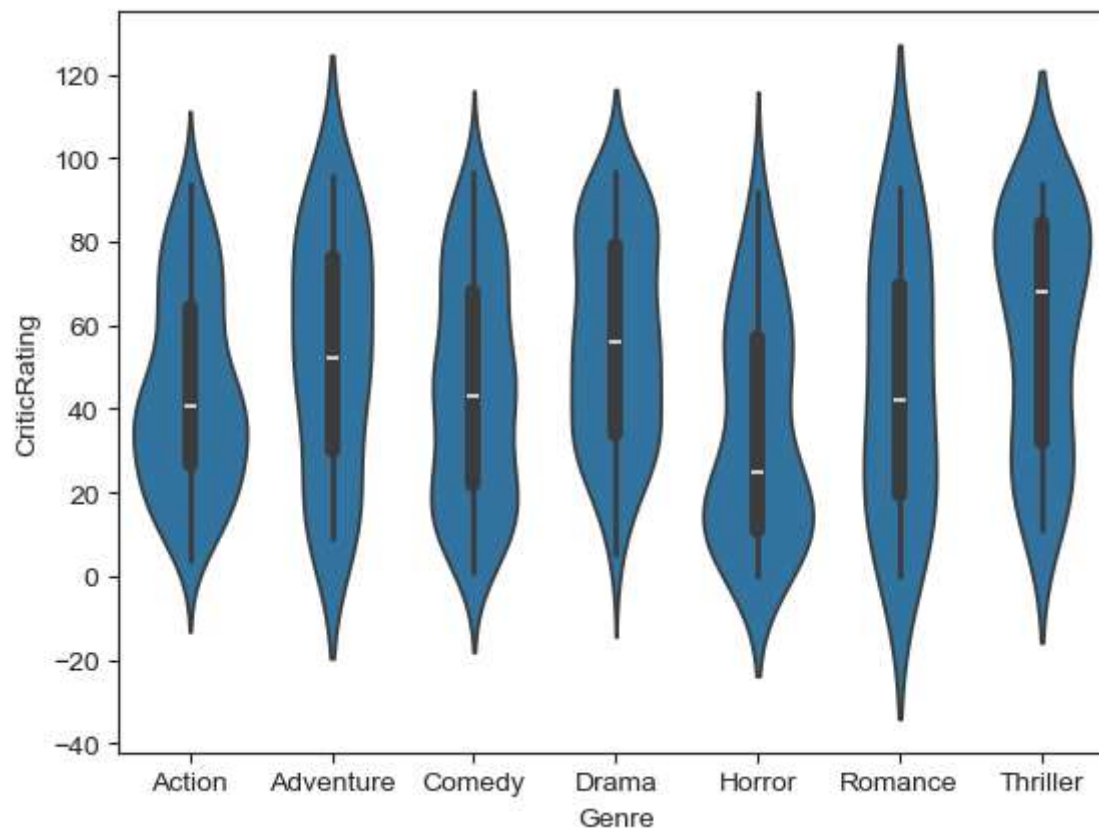
```
In [65]: w = sns.boxplot(data = movies, x = 'Genre', y = 'CriticRating')
```



```
In [66]: w = sns.boxenplot(data = movies, x = 'Genre', y = 'CriticRating')
```



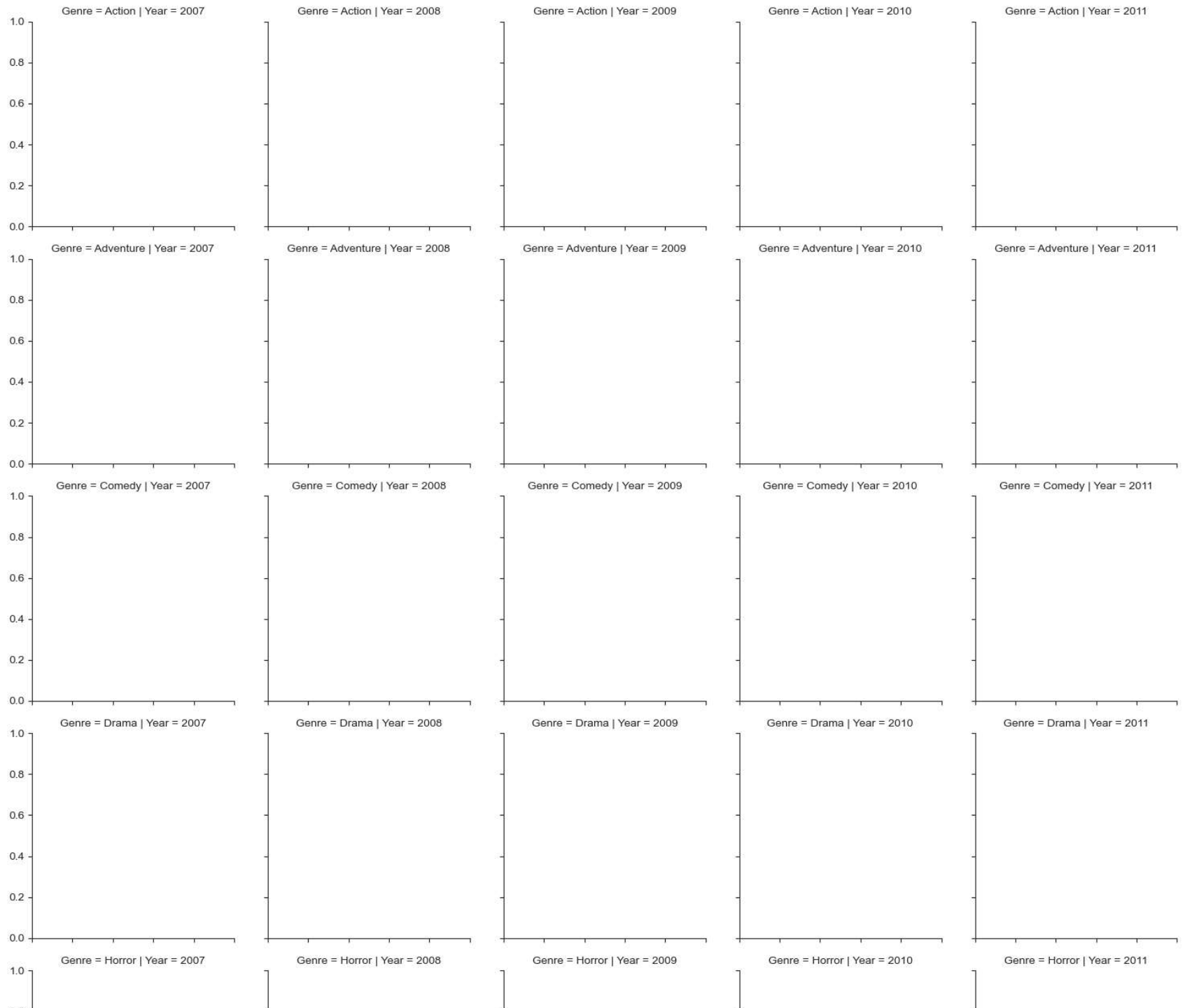
```
In [67]: w = sns.violinplot(data = movies, x = 'Genre', y = 'CriticRating')
```

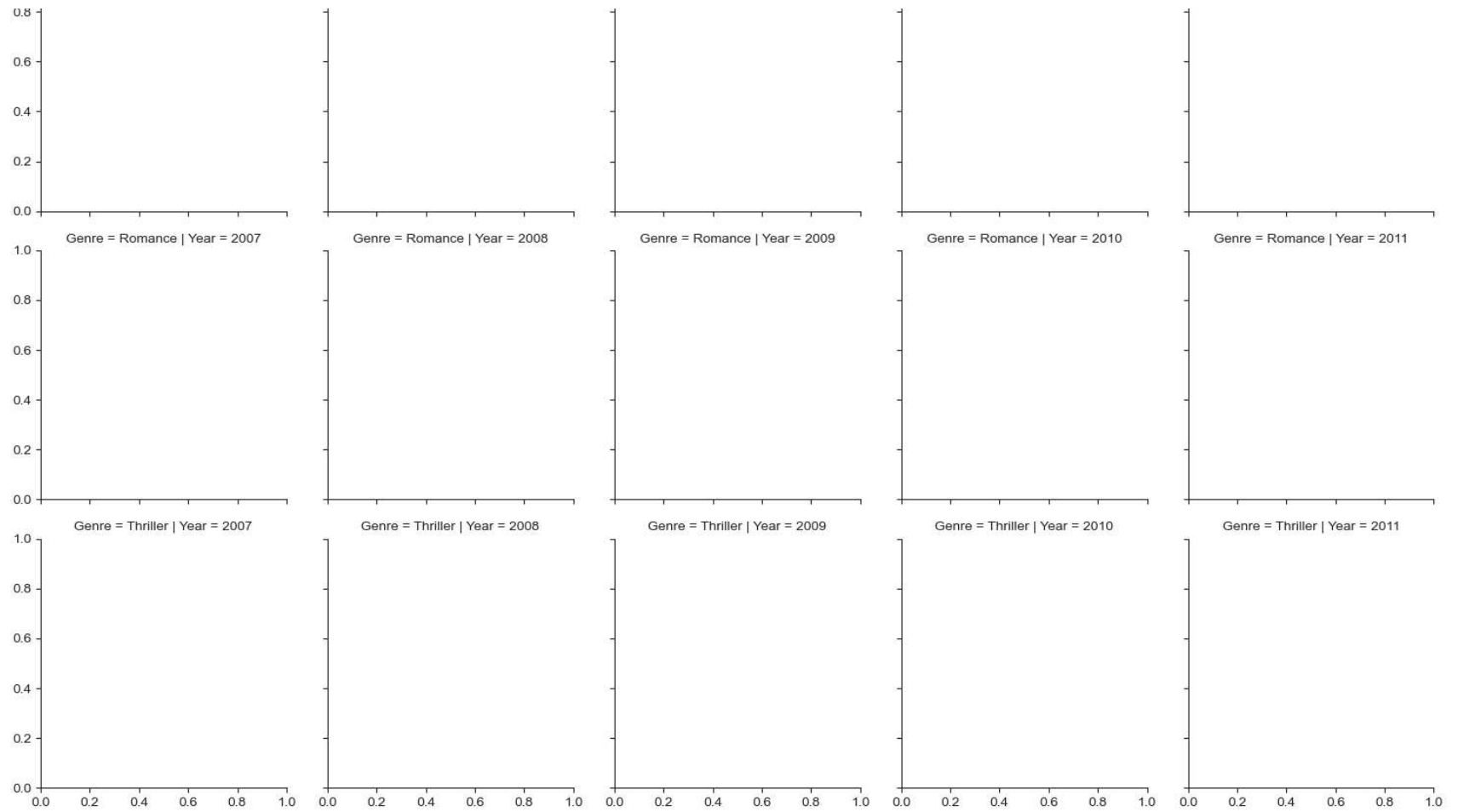


```
In [ ]: #26thNov2025
```

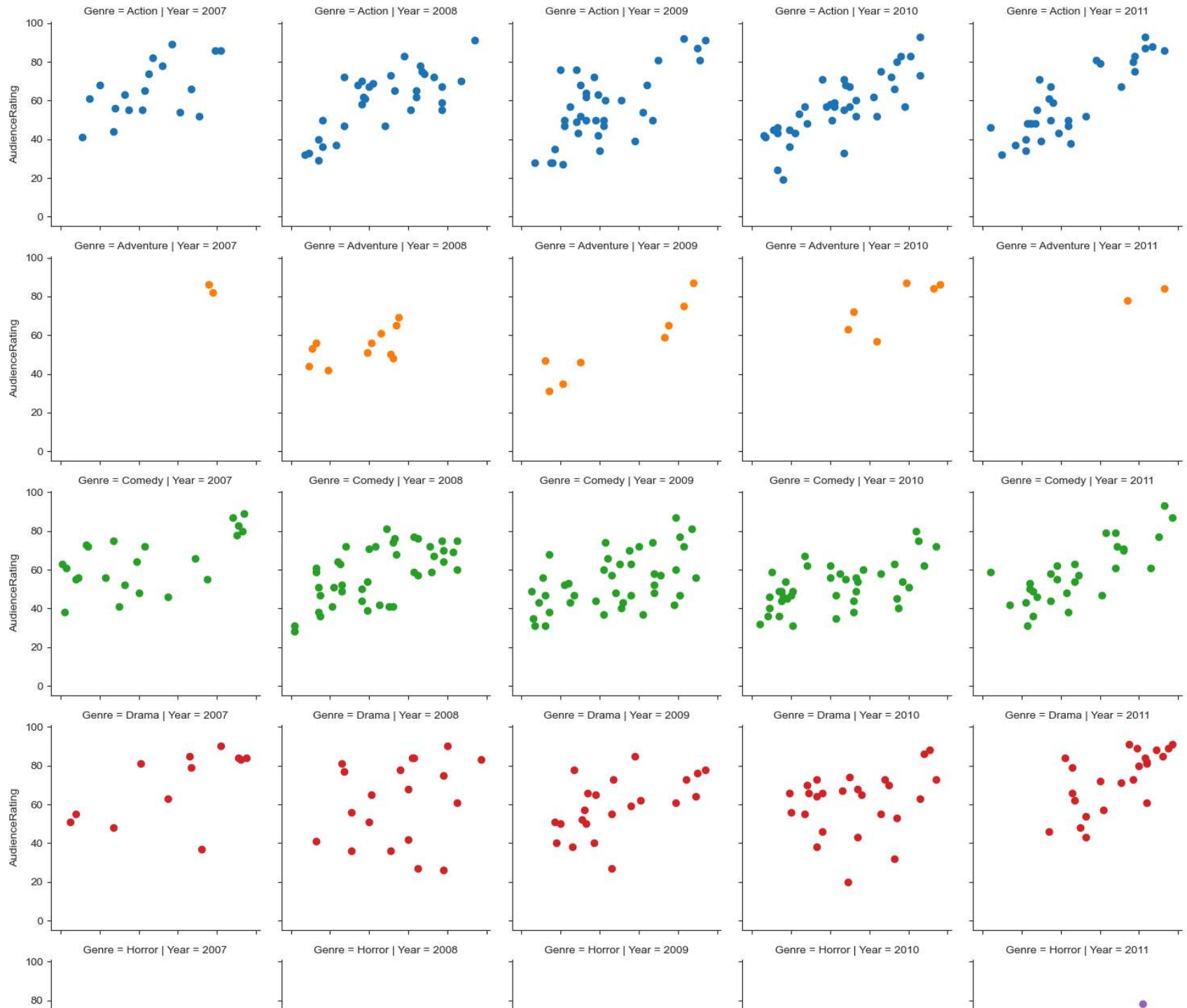
```
In [ ]: #Creating a Facet Grid
```

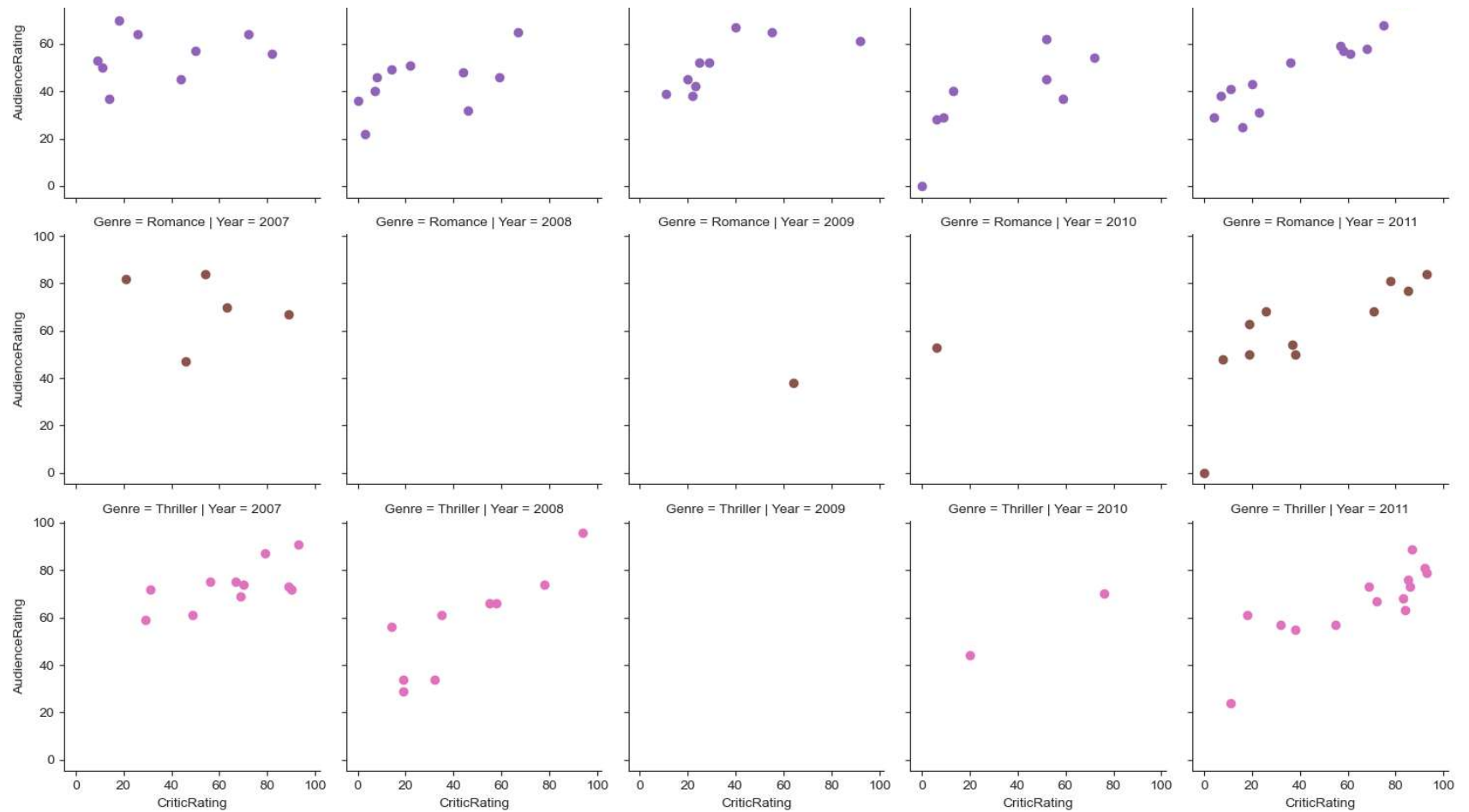
```
In [69]: g = sns.FacetGrid(movies, row = 'Genre', col = 'Year', hue = 'Genre')
```



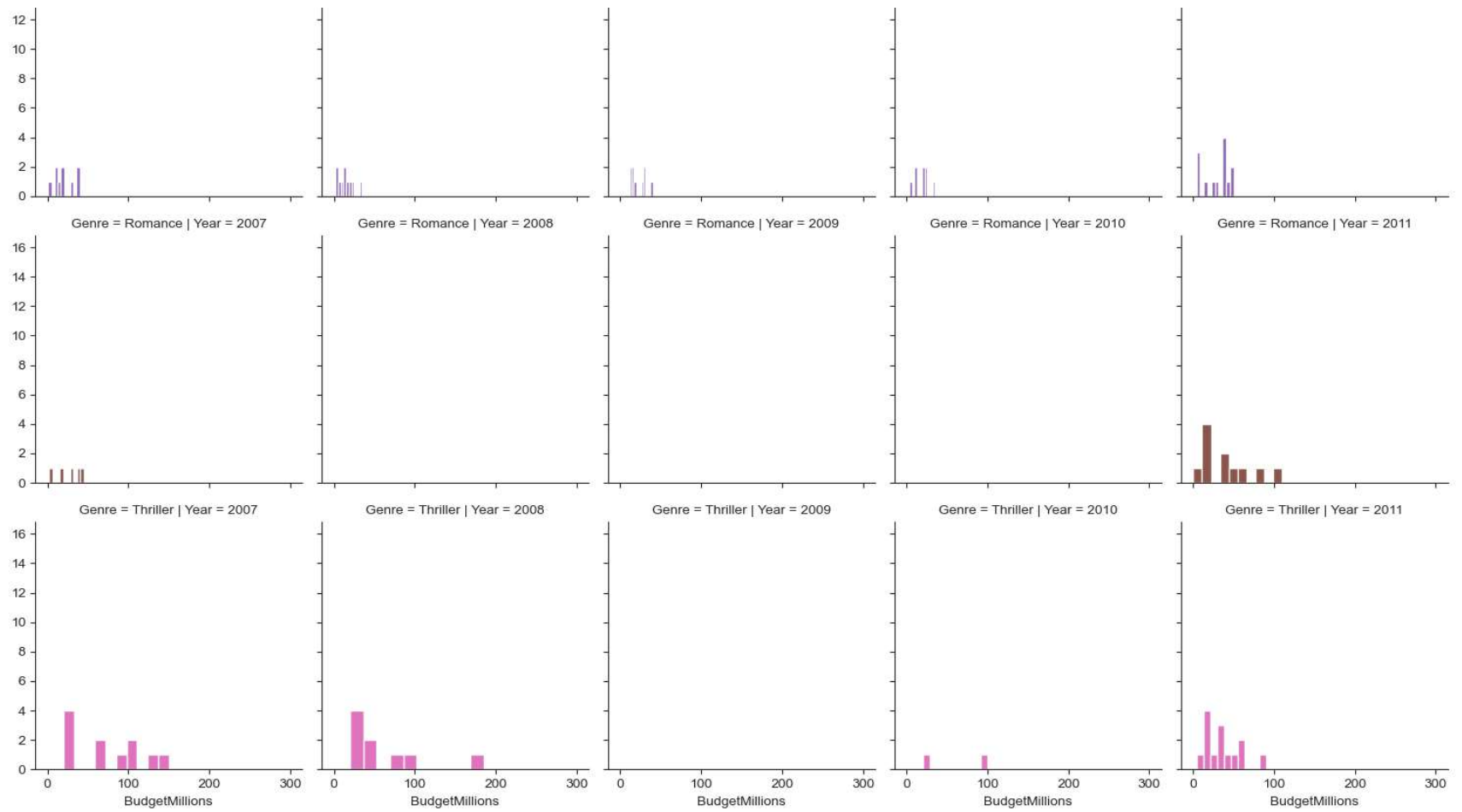


```
In [72]: g = sns.FacetGrid(movies, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.scatter, 'CriticRating', 'AudienceRating')    #Scatter plots are mapped in facet grid
```



```
In [74]: g = sns.FacetGrid(movies, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.hist, 'BudgetMillions')      #Histogram are mapped in facet grid
```

In []:

In []: